

Course Outline: Debugging with Python

Title: Debugging with Python
Learnpack: + Debugging with Python
Prerequisite: Week 10 Day 30 - Managing Errors
Target Audience: Beginner developers building FastAPI backends, familiar with Python fundamentals, error handling, and basic backend development
Total Lessons: 12
Format: Mixed (17% hands-on)

Course Description

This course teaches systematic debugging techniques for Python applications, with a focus on backend development using FastAPI. Students learn to read Python tracebacks, use debugging tools like pdb and IDE debuggers, and apply logging strategies to identify and fix bugs efficiently. The course bridges error handling (preventing crashes) with unit testing (preventing bugs), teaching students how to diagnose issues methodically before they reach production.

By the end of this course, students will be able to interpret Python error messages, use debuggers to step through code execution, apply logging to trace program flow, and debug common FastAPI backend issues including endpoint errors and database problems.

Course Philosophy

- This course emphasizes:
- **Evidence over assumptions:** Use debugging tools to observe what code actually does, not what you think it does
 - **Systematic investigation:** Follow a reproducible process (reproduce, isolate, fix, verify) rather than random changes
 - **Learning from errors:** Every bug is an opportunity to understand your code better and prevent similar issues
 - **Human-in-the-loop debugging:** AI tools can help, but understanding how to debug manually makes you a stronger developer
-

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - Understanding Python Errors	2
02 - Debugging Tools and Techniques	3
03 - Debugging Backend Applications	3

Section	Lessons
04 - Assessment	2
05 - Conclusion	1
Total	12

Section 00: Welcome

00.0 Welcome to Python Debugging

Learning Objectives:

- Understand why systematic debugging skills are essential for backend developers
- Recognize the relationship between error handling, debugging, and testing
- Preview the debugging tools and techniques covered in this course

Content Outline:

- The problem: You've built FastAPI backends, but when something breaks, how do you find out why?
- Why debugging matters: The difference between "it doesn't work" and "I know exactly what's wrong"
- Course journey: From reading error messages to using professional debugging tools
- What you'll build: Practical skills for diagnosing backend issues quickly

Transitions: In the previous lesson, you learned how to handle errors gracefully using try/catch/finally. That keeps your application from crashing. This course teaches you how to find and fix the bugs that cause those errors in the first place. By the end, you'll have systematic debugging skills that prepare you for the unit testing lesson coming next.

Key Principles:

- Debugging is observation, not guessing
- Every bug leaves a trail of evidence
- Good debugging saves time in the long run
- The debugging process is reproducible and teachable

Examples:

- A FastAPI endpoint returns 500 errors for some users but not others. How do you figure out why?
 - Your database query works in testing but fails in production. What's different?
 - Code that worked yesterday suddenly breaks today. Where do you start looking?
-

Section 01: Understanding Python Errors

01.0 Reading Tracebacks and Common Error Types

Learning Objectives:

- Read and interpret Python tracebacks to identify where errors occur

- Recognize common Python error types (NameError, TypeError, AttributeError, KeyError, IndexError, ValueError)
- Distinguish between syntax errors, runtime errors, and logical errors
- Trace the execution path that led to an error

Content Outline:

- Anatomy of a Python traceback: reading from bottom to top
- The error message: type, value, and what they tell you
- Stack frames: understanding the call chain
- Common error types and their typical causes:
 - NameError: variable doesn't exist
 - TypeError: wrong type for operation
 - AttributeError: accessing non-existent attributes
 - KeyError: accessing non-existent dictionary keys
 - IndexError: list index out of range
 - ValueError: correct type but wrong value
- Syntax errors vs runtime errors vs logical errors
- Line numbers: where the error was detected vs where the bug is

Transitions: You've already learned about try/catch for handling errors. Now you'll learn what those error messages are actually telling you. Understanding tracebacks is the first step in debugging—you need to read the evidence before you can follow the trail.

Key Principles:

- The traceback shows you the journey, not just the crash site
- Read from bottom (where it crashed) to top (where the journey started)
- The error type tells you what went wrong; the traceback tells you where
- The last line of the traceback is not always where the bug is

Examples:

- A KeyError in your FastAPI endpoint: The traceback shows it happens when accessing `user["email"]`, but the real problem is that the database returned a user object without an email field
- A TypeError saying "can't concatenate str and int": The error happens in a template, but the bug is in the endpoint that passed a string where a number was expected
- An AttributeError accessing `.name` on None: The traceback points to line 45, but the bug is on line 38 where the function returned None instead of a user object

01.1 From Error Message to Root Cause

Learning Objectives:

- Apply systematic thinking to trace errors from symptom to cause
- Identify the difference between where an error appears and where it originates
- Recognize patterns in error messages that indicate specific problem categories
- Practice the "5 Whys" technique for debugging

Content Outline:

- Symptoms vs causes: the error message is what broke, not why it broke
- Following the data backward: what values led to this error?
- Common patterns:
 - "NoneType has no attribute X" → something returned None unexpectedly
 - "list index out of range" → list shorter than expected
 - "unhashable type: dict" → trying to use mutable type as dictionary key
- The "5 Whys" technique applied to bugs
- When the bug is not where the error happens: data flow analysis
- Environmental factors: database state, API responses, user input

Transitions: Now that you can read tracebacks, you'll learn to follow them backward to find the actual bug. This analytical thinking is essential before you start using debugging tools—you need to know what questions to ask.

Key Principles:

- Ask "why did this value cause an error?" not just "what is the error?"
- The bug is usually upstream from where the error appears
- Data tells the story: trace values backward through your code
- Understanding the error pattern helps you recognize similar bugs faster

Examples:

- A login endpoint returns "list index out of range" when checking `roles[0]`. The error says the list is empty. Why? The database query returned a user with no roles assigned. Why? The registration function never called the code to assign the default role. That's the bug.
- An API call raises `AttributeError: 'NoneType' object has no attribute 'json'`. The error says the response is None. Why? The external API returned a 500 error instead of 200. Why? Your authentication header was malformed. That's the bug.
- A calculation raises `ValueError: "invalid literal for int()"`. The error says you passed a non-numeric string. Why? User input wasn't validated. Why? The frontend form didn't enforce numeric input and the backend didn't check. That's the bug.

Section 02: Debugging Tools and Techniques

02.0 The Python Debugger: pdb and IDE Tools

Learning Objectives:

- Understand what a debugger does and when to use it
- Use Python's built-in pdb debugger to pause and inspect code execution
- Navigate through code with debugger commands (step, next, continue)
- Compare pdb with IDE debugging features and choose the appropriate tool

Content Outline:

- What is a debugger? Pausing time to inspect your program

- Python's pdb module: the built-in debugging tool
- Setting breakpoints with `breakpoint()` (Python 3.7+) or `import pdb; pdb.set_trace()`
- Essential pdb commands:
 - `n` (next): execute the current line
 - `s` (step): step into function calls
 - `c` (continue): run until next breakpoint
 - `l` (list): show surrounding code
 - `p variable` (print): inspect variable values
 - `w` (where): show stack trace
 - `q` (quit): exit debugger
- IDE debugging features: VS Code, PyCharm, Cursor
 - Visual breakpoints (click to add)
 - Watch variables (automatic display)
 - Step through controls (buttons)
 - Call stack visualization
- When to use pdb vs IDE debugger vs print statements

Transitions: Reading tracebacks tells you where code failed after the fact. Debuggers let you watch code execute step-by-step before it fails, so you can see exactly what's happening. This is the most powerful debugging tool you'll learn.

Key Principles:

- Debuggers let you stop time and inspect your program mid-execution
- Stepping through code reveals what's actually happening, not what you think is happening
- Visual IDE debuggers are easier for beginners; pdb is essential for remote servers
- Use debuggers when the bug is hard to isolate with print statements alone

Examples:

- Your function should return a list but sometimes returns an empty list. Set a breakpoint inside the function, step through the loop, and watch when the list stops getting values added.
- A calculation gives wrong results. Use the debugger to pause before the calculation, inspect all the input variables, step through each operation, and see where the math goes wrong.
- An if statement behaves unexpectedly. Set a breakpoint on the if line, inspect the condition variables, and see why the condition evaluates differently than you expected.

02.1 Using pdb to Fix Bugs

Challenge Prompt: A function calculates user discounts based on their purchase history, but it returns incorrect discount percentages. Use pdb to step through the function, inspect variables at each step, and identify where the calculation goes wrong.

Solution Code:

```
def calculate_discount(purchase_count, total_spent):  
    """Calculate discount percentage based on purchase history"""  
    discount = 0
```

```

    if purchase_count >= 10:
        discount += 10

    if total_spent >= 1000:
        discount += 15
    elif total_spent >= 500:
        discount += 10
    elif total_spent >= 100:
        discount += 5

    # Cap discount at 25%
    if discount > 25:
        discount = 25

    return discount

# Test cases
print(f"5 purchases, $50: {calculate_discount(5, 50)}%") # Expected: 0%
print(f"12 purchases, $150: {calculate_discount(12, 150)}%") # Expected: 15%
print(f"15 purchases, $1200: {calculate_discount(15, 1200)}%") # Expected: 25%

```

Challenge Code:

```

def calculate_discount(purchase_count, total_spent):
    """Calculate discount percentage based on purchase history"""
    discount = 0

    # Bug: Logic error in discount calculation
    if purchase_count >= 10:
        discount += 10

    if total_spent >= 1000:
        discount += 15
    # Missing elif - adds multiple discounts incorrectly
    if total_spent >= 500:
        discount += 10
    if total_spent >= 100:
        discount += 5

    # Bug: Wrong comparison operator
    if discount >= 25:
        discount = 25

    return discount

# TODO: Use pdb to debug this function
# 1. Add breakpoint() at the start of the function
# 2. Run the test cases
# 3. Step through each condition
# 4. Watch how discount value changes
# 5. Find where the logic goes wrong

```

```
# 6. Fix the bugs

# Test cases - these will give wrong results
print(f"5 purchases, $50: {calculate_discount(5, 50)}%") # Should be 0%, returns 5%
print(f"12 purchases, $150: {calculate_discount(12, 150)}%") # Should be 15%, returns 25%
print(f"15 purchases, $1200: {calculate_discount(15, 1200)}%") # Should be 25%, returns 30% → 25%
```

02.2 Logging Strategies for Python Applications

Learning Objectives:

- Understand the difference between print debugging and proper logging
- Use Python's logging module to track program execution
- Configure appropriate log levels (DEBUG, INFO, WARNING, ERROR, CRITICAL)
- Apply logging effectively in FastAPI applications

Content Outline:

- Why logging beats print statements:
 - Logs can be turned on/off without code changes
 - Logs include timestamps and severity levels
 - Logs can go to files, not just console
 - Logs don't clutter production output
- Python's logging module basics:
 - `import logging`
 - `logging.debug()`, `logging.info()`, `logging.warning()`, `logging.error()`, `logging.critical()`
 - Setting log level: `logging.basicConfig(level=logging.DEBUG)`
- Log levels and when to use each:
 - DEBUG: detailed diagnostic information
 - INFO: confirmation that things are working
 - WARNING: something unexpected but not an error
 - ERROR: a serious problem occurred
 - CRITICAL: the program may not be able to continue
- Logging in FastAPI:
 - Using Python's logging with FastAPI endpoints
 - What to log: request data, validation errors, database queries, external API calls
 - What not to log: passwords, tokens, sensitive user data
- Structured logging for production: JSON logs for parsing

Transitions: Debuggers are perfect for local development, but what about bugs that only happen on the server? That's where logging comes in. Good logging practices let you "debug" production issues by analyzing logs after the fact.

Key Principles:

- Logs are the flight recorder of your application
- Log at appropriate levels: not everything is an error
- Never log sensitive data—it's a security risk
- Logs should tell a story: what happened, when, and with what data

Examples:

- A FastAPI endpoint sometimes fails. Add logging before the database query, after the query, and before returning the response. The logs reveal that failures happen when the query returns empty results, and the code tries to access `results[0]` without checking if the list is empty.
 - An external API call fails unpredictably. Add logging to capture the request URL, headers, payload, and response code. The logs show that failures happen when the request payload exceeds 1MB, hitting the API's undocumented size limit.
 - A calculation produces wrong results for some users. Add logging to capture input values. The logs reveal that the bug only affects users with null values in optional fields—the code doesn't handle nulls properly.
-

Section 03: Debugging Backend Applications

03.0 Debugging FastAPI Endpoints

Learning Objectives:

- Apply debugging techniques specifically to FastAPI endpoint problems
- Use request/response inspection to diagnose API issues
- Debug validation errors, serialization problems, and endpoint logic
- Integrate debugging tools with API testing tools like Postman

Content Outline:

- Common FastAPI debugging scenarios:
 - Endpoint returns 422 (validation errors)
 - Endpoint returns 500 (unhandled exceptions)
 - Endpoint returns correct status but wrong data
 - Endpoint times out or is very slow
- Debugging request data:
 - Log the incoming request body, headers, query params
 - Check what Pydantic validation rejected
 - Verify authentication tokens and permissions
- Debugging response data:
 - Inspect what your endpoint is actually returning
 - Check serialization: are your models converting correctly?
 - Verify response status codes match the situation
- Using Postman/Thunder Client with debugging:
 - Send test requests while debugging
 - Inspect actual request/response in network tools
 - Save problematic requests to reproduce bugs
- FastAPI's automatic docs for debugging: `/docs` endpoint shows request/response schemas

Transitions: Now you'll apply debugging techniques to the backend code you've been writing. FastAPI introduces specific challenges—validation, serialization, async code—that require adapted debugging strategies.

Key Principles:

- API bugs usually involve data transformation: what went in vs what came out
- Validation errors mean Pydantic rejected the data—log what was rejected
- 500 errors hide the real problem—add try/catch with logging
- Test with real HTTP requests, not just Python functions

Examples:

- Endpoint returns 422. Check the FastAPI docs at [/docs](#)—it shows exactly which field failed validation and why. Add logging to capture the rejected payload for investigation.
 - Endpoint returns 500 with no helpful error. Wrap the endpoint logic in try/except, log the full exception with `logging.exception()`, and the logs reveal a KeyError when accessing a dictionary key that doesn't always exist.
 - Endpoint returns data but it's wrong. Use a debugger with a Postman request: set breakpoint in the endpoint, send request from Postman, step through the code to see where the data gets transformed incorrectly.
-

03.1 Debugging Database Operations

Learning Objectives:

- Diagnose common database-related bugs in Python applications
- Debug SQL queries, connection issues, and ORM problems
- Use logging to trace database operations
- Identify performance issues through query analysis

Content Outline:

- Common database debugging scenarios:
 - Query returns no results when it should
 - Query returns wrong results
 - Database connection fails
 - ORM generates wrong SQL
 - Query is too slow
- Debugging SQL queries:
 - Log the actual SQL being executed
 - Test queries directly in database client
 - Check for typos in table/column names
 - Verify WHERE clause logic
- Connection issues:
 - Check connection string format
 - Verify database is running and accessible
 - Check authentication credentials
 - Review firewall/network settings

- ORM debugging (SQLModel, SQLAlchemy):
 - Enable SQL query logging to see generated queries
 - Use `.all()` vs `.first()` correctly
 - Check relationship loading (N+1 query problem)
 - Verify model definitions match database schema
- Query performance debugging:
 - Log query execution time
 - Identify slow queries
 - Check for missing indexes
 - Review whether you're fetching too much data

Transitions: Your FastAPI endpoints often interact with databases, and database bugs can be tricky—the error might appear in your Python code but originate in the database layer. Understanding database debugging connects your backend logic to your data persistence.

Key Principles:

- Log the actual SQL to see what the database receives
- Test database queries independently of your Python code
- Connection errors usually mean configuration problems, not code bugs
- Performance problems are bugs too—query optimization is debugging

Examples:

- A query returns no results. Log the generated SQL, copy it into your database client, run it directly—discover that the table name has a typo in your model definition.
- Database connection fails with "authentication error." Add logging for the connection string (with password redacted), discover you're using the wrong database name.
- An endpoint is slow. Add logging to measure query time, discover one query takes 2 seconds. Log the SQL, see it's fetching 10,000 rows when it only needs 10—add a LIMIT clause.

03.2 Fix the Backend Bug

Challenge Prompt: A FastAPI endpoint retrieves user information and returns it with a calculated total spending value. The endpoint sometimes returns incorrect totals and sometimes crashes with a database error. Use debugging tools to find and fix both bugs.

Solution Code:

```
from fastapi import FastAPI, HTTPException
from typing import Optional
import logging

app = FastAPI()
logging.basicConfig(level=logging.DEBUG)

# Mock database
users_db = [
    {"id": 1, "name": "Alice", "email": "alice@example.com"},
```

```

        {"id": 2, "name": "Bob", "email": "bob@example.com"},
    ]

    purchases_db = [
        {"user_id": 1, "amount": 50.00},
        {"user_id": 1, "amount": 75.50},
        {"user_id": 2, "amount": 100.00},
    ]

    @app.get("/users/{user_id}/summary")
    def get_user_summary(user_id: int):
        logging.debug(f"Fetching user {user_id}")

        # Find user
        user = next((u for u in users_db if u["id"] == user_id), None)
        if not user:
            logging.warning(f"User {user_id} not found")
            raise HTTPException(status_code=404, detail="User not found")

        # Calculate total spending
        user_purchases = [p for p in purchases_db if p["user_id"] == user_id]
        total_spent = sum(p["amount"] for p in user_purchases)

        logging.info(f"User {user_id} has {len(user_purchases)} purchases, total:
        ${total_spent}")

        return {
            "user": user,
            "total_purchases": len(user_purchases),
            "total_spent": round(total_spent, 2)
        }

```

Challenge Code:

```

from fastapi import FastAPI, HTTPException
import logging

app = FastAPI()
logging.basicConfig(level=logging.DEBUG)

# Mock database
users_db = [
    {"id": 1, "name": "Alice", "email": "alice@example.com"},
    {"id": 2, "name": "Bob", "email": "bob@example.com"},
]

purchases_db = [
    {"user_id": 1, "amount": 50.00},
    {"user_id": 1, "amount": 75.50},
    {"user_id": 2, "amount": 100.00},
]

```

```
@app.get("/users/{user_id}/summary")
def get_user_summary(user_id: int):
    # Bug 1: No logging to trace execution

    # Bug 2: Will crash if user not found
    user = next(u for u in users_db if u["id"] == user_id)

    # Bug 3: Wrong filter - doesn't match types correctly
    user_purchases = [p for p in purchases_db if p["user_id"] == str(user_id)]

    # Bug 4: Will crash if user_purchases is empty
    total_spent = sum(p["amount"] for p in user_purchases)

    return {
        "user": user,
        "total_purchases": len(user_purchases),
        "total_spent": total_spent
    }

# TODO: Debug this endpoint
# 1. Test with user_id=1, user_id=2, and user_id=99
# 2. Use pdb to step through the function
# 3. Add logging to trace execution
# 4. Fix the bugs:
#     - Handle user not found gracefully
#     - Fix the type mismatch in user_purchases filter
#     - Ensure total_spent works even with no purchases
#     - Add proper error handling
```

Section 04: Assessment

04.0 Systematic Debugging Methodology

Learning Objectives:

- Apply a structured debugging process to any bug
- Document bugs effectively for yourself and your team
- Prevent bugs from recurring through root cause analysis
- Develop debugging habits that improve code quality

Content Outline:

- The systematic debugging process:
 1. **Reproduce:** Can you make the bug happen reliably?
 2. **Isolate:** What's the minimum code needed to trigger the bug?
 3. **Understand:** What is the code actually doing vs what it should do?
 4. **Fix:** Change the code to correct the behavior
 5. **Verify:** Test that the fix works and didn't break anything else
 6. **Prevent:** How do you make sure this bug never happens again?
- Reproduction strategies:

- Gather exact inputs that trigger the bug
 - Document environment conditions
 - Create a minimal test case
- Isolation techniques:
 - Comment out code sections
 - Test components independently
 - Use binary search: disable half the code, see if bug persists
- Understanding with debugging tools:
 - Use debugger to watch code execute
 - Add logging to trace data flow
 - Compare actual vs expected behavior at each step
- Documentation habits:
 - Write down what you tried
 - Note which approaches didn't work
 - Explain the fix in comments or commit messages
- Prevention through testing:
 - Write a test that catches this bug
 - Add validation to prevent bad inputs
 - Improve error messages for faster diagnosis next time

Transitions: You've learned the tools—tracebacks, debuggers, logging. Now you'll learn the process: how to approach any bug systematically. This methodology works for any programming language and any problem.

Key Principles:

- Debugging is a skill that improves with practice
- Systematic approaches are faster than random changes
- Good debugging makes you a better programmer
- Every bug teaches you something about your code

Examples:

- Bug report: "The login sometimes doesn't work." Reproduce: Try 10 logins—5 work, 5 fail. Pattern? Failures happen when users type email with uppercase letters. Isolate: Test the email comparison code. Understand: Email comparison is case-sensitive. Fix: Convert emails to lowercase before comparison. Verify: All 10 logins now work. Prevent: Add a test for case-insensitive email login.
- Bug report: "API returns 500 error randomly." Reproduce: Make 100 requests, 3 fail. Isolate: Log all failed requests—they all have missing query parameters. Understand: Code doesn't validate required parameters. Fix: Add validation that returns 422 if parameters missing. Verify: Missing parameters now return 422 with clear error message. Prevent: Add integration tests for required parameters.
- Bug report: "Calculation gives wrong results for large numbers." Reproduce: Works for inputs < 1000, fails for inputs > 1000. Isolate: Test just the calculation function. Understand: Using integer division instead of float division. Fix: Change `/` to ensure float result. Verify: Large numbers now calculate correctly. Prevent: Add test cases for boundary values.

04.1 Debugging Skills Assessment 🧠

Learning Objectives:

- Demonstrate understanding of Python debugging concepts
- Apply debugging knowledge to realistic scenarios
- Identify appropriate debugging tools for different situations
- Evaluate debugging approaches for effectiveness

Question Format: Scenario-based (7 questions)

Questions:

Question 1: You get this error when running your FastAPI endpoint:

```
KeyError: 'email'  
File "/app/endpoints.py", line 45, in get_user  
    return {"email": user["email"]}
```

What is the most likely cause?

- A) The database table doesn't have an email column
- B) The user dictionary returned from the database doesn't contain an 'email' key
- C) The endpoint URL is wrong
- D) Python doesn't recognize the string 'email'

Correct Answer: B

Question 2: Your function calculates the wrong result but doesn't raise any errors. Which debugging approach is most effective?

- A) Add print statements at the start and end of the function
- B) Use pdb to step through the function line by line and inspect variables
- C) Rewrite the entire function from scratch
- D) Add try/except around the entire function

Correct Answer: B

Question 3: A bug only happens in production, never in your local development environment. What should you do first?

- A) Deploy your code multiple times until you can reproduce it locally
- B) Check the production logs for error details and patterns
- C) Ask users to stop using the feature until you fix it
- D) Use pdb to debug the production server

Correct Answer: B

Question 4: Your FastAPI endpoint returns 422 Unprocessable Entity. What does this mean and how should you debug it?

- A) The server crashed; check the server logs
- B) The database is down; check database connection

- C) Pydantic validation failed; check FastAPI docs at [/docs](#) to see which field was rejected
- D) The endpoint doesn't exist; check your route definitions

Correct Answer: C

Question 5: You've set a breakpoint in your function using `breakpoint()`, but when you run the code, it doesn't stop. What might be wrong?

- A) You need to import `pdb` first
- B) Breakpoints only work in production, not locally
- C) The code path with the breakpoint is never executed
- D) Python version is too old to support `breakpoint()`

Correct Answer: C

Question 6: A database query returns an empty list when you expect results. What's the best debugging sequence?

- A) Rewrite the query, restart the database, check if the problem is fixed
- B) Log the generated SQL, run it directly in a database client, verify the data exists
- C) Delete the database and recreate it from scratch
- D) Change the ORM library you're using

Correct Answer: B

Question 7: When should you use `logging.error()` vs `logging.debug()`?

- A) `logging.error()` for syntax errors, `logging.debug()` for logical errors
- B) `logging.error()` for serious problems that need attention, `logging.debug()` for detailed diagnostic information during development
- C) They do the same thing, use either one
- D) `logging.error()` stops the program, `logging.debug()` doesn't

Correct Answer: B

Evaluation Criteria:

- 7/7: Excellent understanding of debugging concepts and tools
- 5-6/7: Good grasp of debugging; review specific weak areas
- 3-4/7: Basic understanding; practice systematic debugging more
- 0-2/7: Needs to review core debugging concepts and practice with tools

Score Interpretation:

- **Excellent (7/7):** You have strong debugging skills and understand when to use different tools. You're ready for unit testing and TDD concepts.
- **Good (5-6/7):** You understand most debugging concepts. Review the questions you missed and practice those specific techniques.
- **Basic (3-4/7):** You've learned the core concepts but need more hands-on practice. Work through the debugging exercises again and apply these techniques to your own code.

- **Needs Review (0-2/7):** Debugging concepts need reinforcement. Revisit the lessons on reading tracebacks and using debugging tools, and practice with simple examples before moving to complex scenarios.
-

Section 05: Conclusion

05.0 Next Steps in Your Debugging Journey

Content Outline:

- **Course recap:** You learned to read Python tracebacks, use debuggers (pdb and IDE tools), apply logging strategies, and debug FastAPI backends and database operations. You now have a systematic debugging process that works for any bug.
- **Connection to bigger picture:** Debugging connects to what came before (error handling keeps apps running) and what comes next (unit testing prevents bugs from reaching production). Together, error handling + debugging + testing form your quality assurance toolkit. Professional developers spend significant time debugging—these skills make you faster and more confident.
- **Next steps and continued learning:**
 - Practice debugging on your own projects—every bug is a learning opportunity
 - In the next lesson (Unit Testing), you'll learn to write tests that catch bugs automatically
 - As you build more complex applications, your debugging skills will be essential for maintaining them
 - Learn advanced debugging: remote debugging, debugging async code, profiling for performance
- **Resources for further exploration:**
 - Python's pdb documentation: <https://docs.python.org/3/library/pdb.html>
 - FastAPI debugging guide: <https://fastapi.tiangolo.com/tutorial/debugging/>
 - The Debugging Mindset by Gerald Weinberg
 - "Debugging: The 9 Indispensable Rules" by David Agans
- **Encouragement and celebration:** Debugging can be frustrating, but it's also deeply satisfying—there's nothing quite like finally understanding why something broke and fixing it. You've gained skills that make you more independent and capable as a developer. Every bug you solve makes you better at solving the next one. You're now equipped to tackle problems methodically instead of randomly, and that's the mark of a professional developer. Keep practicing these techniques, and you'll find yourself debugging faster and with more confidence every time.

Note: This is conclusion only - no theory, exercises, or assessment.
